

Examencompletop3.pdf



elver_g4larg4



Sistemas Concurrentes y Distribuidos



2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada



MÁSTER EN

Inteligencia Artificial & Data Management

MADRID

Formamos
talento para un futuro
Sostenible

saber más



**LLÉVATE 2 CLASES
PRÁCTICAS GRATIS**

Elige entre el permiso A1 o B

APROVECHA LA
SÚPER OFERTA



Examen SCD de la práctica 3

Enunciados completos
+
pseudocódigo

att:elver_g4larg4

ESCANÉA EL QR
Llévate 2 clases prácticas
totalmente **Gratis**



Ejercicio 1:

Construir un programa MPI cuyo nombre deberá ser `filo_ex.cpp`, con operaciones de paso de mensaje síncronas, modificando el problema de la cena de los filósofos con camarero de la siguiente manera:

El filósofo nº 4 (identificador 8) es un auténtico “malafollá” y la lía siempre durante la comida originando mal ambiente. El resto de filósofos (“amables”) no quieren comer con él y prefiere saltarse la comida salvo que estén hambrientos. Cada vez que comen dichos filósofos “amables”, pasan a estado saciado y podrían saltarse la comida de la siguiente iteración. Si se saltan una comida, pasan al estado hambriento. Cualquier filósofo “amable”, después de enviar el mensaje para sentarse, debe esperar a recibir un mensaje del camarero donde dicho proceso camarero le indica si está sentado o no el filósofo “malafollá”. Si no está sentado el filósofo “malafollá”, come normalmente. Si está sentado el filósofo “malafollá”, su acción depende de su estado. Si estaba hambriento, come normalmente y si estaba saciado, va directamente al paso de levantarse de la mesa (sin interactuar con los procesos tenedores ni comer), pasando entonces a estado hambriento.

El filósofo “malafollá” come siempre normalmente porque, como buen malaje, los demás le importan un bledo.

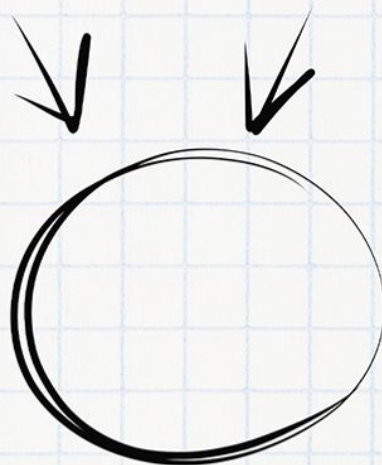
Los filósofos “amables” comienzan el programa en estado hambriento.

Imagínate aprobando el examen

Necesitas tiempo y concentración

Planes	 PLAN TURBO	 PLAN PRO	 PLAN PRO+
 Descargas sin publi al mes	10 	40 	80 
 Elimina el video entre descargas			
 Descarga carpetas			
 Descarga archivos grandes			
 Visualiza apuntes online sin publi			
 Elimina toda la publi web			
 Precios Anual ☐	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo,
¿Qué nota vas a sacar?



WUOLAH

Sistemas Concurrentes y Dist...



Comparte estos flyers en tu clase y consigue más dinero y recompensas



Banco de apuntes de la

WUOLAH

- 1** Imprime esta hoja
- 2** Recorta por la mitad
- 3** Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes

- 4** Llévate dinero por cada descarga de los documentos descargados a través de tu QR



Ejercicio 2: (pseudocódigo abajo)

En la delegación de hacienda hay cinco mostradores (procesos 0 al 4) para atender a 11 clientes (procesos 5 al 15). Los procesos clientes repiten un bucle de pagar en hacienda y trabajar para poder pagar a hacienda. Los procesos mostradores repiten un ciclo de atender a un cliente. Para gestionar los mostradores, se utiliza un sistema de cola única que gestiona un proceso controlador (proceso 16) que recibe las peticiones de servicio de los clientes y controla la ocupación de cada mostrador para dar paso a dichos clientes. Cuando el proceso controlador recibe la petición de un cliente, busca un mostrador libre y manda un mensaje a dicho cliente indicándole el identificador de dicho proceso mostrador. Entonces, el cliente manda un mensaje al proceso mostrador, espera a realizar el pago, que acaba cuando recibe un mensaje de dicho proceso mostrador y manda un mensaje al controlador para indicar que el mostrador está libre. Construir un programa MPI con operaciones de paso de mensaje síncronas para resolver el problema (todos los procesos son bucles infinitos). Muestra mensajes por pantalla para seguir la traza del programa. El fichero se deberá llamar examenp3.cpp. Esquema del programa en el archivo "programa-ejercicio-2"

LLÉVATE 2 CLASES PRÁCTICAS GRATIS

Elige entre el permiso A1 o B

APROVECHA LA
SÚPER OFERTA



Esquema del programa del ejercicio 2:

Proceso_cliente

```
ssend(controlador,var) {solicitar mostrador}
receive(controlador,n_m) {en n_m se almacena el mostrador}
ssend(n_m,var) {llegada al mostrador}
receive(n_m, var) {trámite finalizado}
ssend(controlador,n_m) {servicio completado}
{retardo aleatorio} {trabajo de sol a sol}
```

Proceso_mostrador

```
receive(ANY,var) {recibir de un cliente}
cli:= proceso emisor del mensaje anterior;
{retardo aleatorio (tiempo del trámite)}
ssend(cli, var) {trámite finalizado}
```

Proceso_controlador

```
Si hay algún mostrador libre (se debe almacenar el estado de cada mostrador)
esperar un mensajes de los clientes (de solicitar mostrador o de servicio completado)
En caso contrario (no hay mostradores libres)
esperar un mensaje de los clientes (sólo de servicio completado)

Si se ha recibido un mensaje de solicitar servicio
mos:= buscar un mostrador libre
actualizar estado mostradores
ssend(emisor,mos) {se indica al cliente que puede ir al proceso mostrador almacenado en la variable mos}
Si se ha recibido un mensaje de servicio completado
actualizar estado mostradores
```

ESCANÉA EL QR
Llévate 2 clases prácticas
totalmente **Gratis**



